2020년 겨울계절학기

네트워크프로그래밍 실습 & 과제 #8

21. 01. 04

#8. Raw Socket

실습 및 과제 #08

실습 내용

- ICMP
- Raw Socket의 특징
- Raw Socket 구현 (과제)

ICMP (Internet Control Message Protocol)

- ICMP 메시지는 주로 IP 동작에서 제어로 사용되거나 오류에 대한 응답으로 만들어짐
- 용도 : 인터넷 통신 상에서 발생한 일반적인 상황에 대한 보고 또는 인터넷 상황 점검
- ICMP를 사용하는 주명령어
  - ping : 상대방 호스트 작동 여부 및 응답 시간 측정
  - tracert : 목적지까지의 라우팅 경로 추적

Raw Socket의 특징

- 프로토콜 헤더를 직접 조작하여 기존에 사용하던 TCP, UDP 소켓보다 세부적인 제어 가능
- 프로토콜 헤더의 구조와 동작 원리에 대한 이해를 필요로 함
- 운영체제마다 생성할 수 있는 Raw Socket의 종류가 다름
- Port가 없어 해당 호스트에 열려 있는 소켓으로 교환하는 모든 메시지를 확인할 수 있음
  - WireShark 같은 프로그램에 사용됨

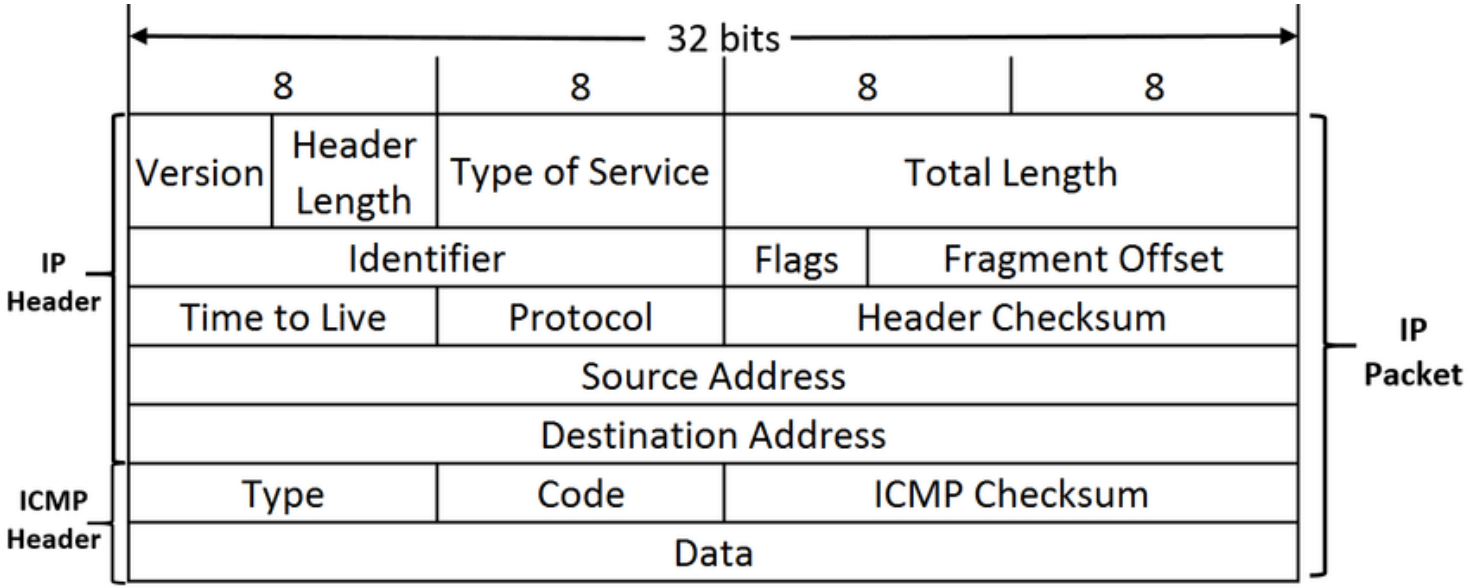
Raw Socket 구현

```c
#include <netinet/ip.h>      // struct ip*
#include <netinet/ip_icmp.h> // stuct icmp*
#include <netdb.h>           // gethostbyname

#define PACKET_LEN 64
#define BUFFER_SIZE 500
#define MAX_LOOP 5
```

```c
int main(int argc, char *argv[])
{
    int ping_sock; // socket discriptor
    struct hostent *dst_net;
    struct sockaddr_in dst_sa;
    struct ip *ip;
    struct icmp *icmp;
    struct timeval tv;
    char send_buf[PACKET_LEN];
    char recv_buf[BUFFER_SIZE];
    u_int dst_ip;
    int request;
    int rcv_len;
    int cnt = 0;
    char str[32];
    fd_set readfd;
    int i;

    if (argc != 2)
    {
        printf("Usage : %s <Address>\n", argv[0]);
        exit(0);
    }
```



- Type (8 bits) : 메시지 유형
- Code (8 bits) : 메시지 유형마다 추가적인 세부정보
- Checksum (16 bits) : 변조 확인
- Data : 가변길이로 type에 따라 필요한 payload가 붙을 수 있음

Raw Socket 구현

```c
#include <netinet/ip.h>      // struct ip*
#include <netinet/ip_icmp.h> // stuct icmp*
#include <netdb.h>           // gethostbyname

#define PACKET_LEN 64
#define BUFFER_SIZE 500
#define MAX_LOOP 5
```

```c
int main(int argc, char *argv[])
{
    int ping_sock; // socket discriptor
    struct hostent *dst_net;
    struct sockaddr_in dst_sa;
    struct ip *ip;
    struct icmp *icmp;
    struct timeval tv;
    char send_buf[PACKET_LEN];
    char recv_buf[BUFFER_SIZE];
    u_int dst_ip;
    int request;
    int rcv_len;
    int cnt = 0;
    char str[32];
    fd_set readfd;
    int i;

    if (argc != 2)
    {
        printf("Usage : %s <Address>\n", argv[0]);
        exit(0);
    }
```

- Type (8 bits) : 메시지 유형
- Code (8 bits) : 메시지 유형마다 추가적인 세부정보

TYPE	CODE	Message
0		Echo Reply
3		Destination Unreachable
	0	Network Unreachable
	1	Host Unreachable
	2	Protocol Unreachable
	6	Destination Network Unknown
	7	Destination Host Unknown
5		Redirect
	0	Redirect Datagram for the Network
	1	Redirect Datagram for the host
8		Echo Request
11		Time Exceeded
	0	Time to Live exceeded in Transit
	1	Fragment Reassembly Time Exceeded

Raw Socket 구현

```c
#include <netinet/ip.h>      // struct ip*
#include <netinet/ip_icmp.h> // stuct icmp*
#include <netdb.h>           // gethostbyname

#define PACKET_LEN 64
#define BUFFER_SIZE 500
#define MAX_LOOP 5
```

```c
int main(int argc, char *argv[])
{
    int ping_sock; // socket discriptor
    struct hostent *dst_net;
    struct sockaddr_in dst_sa;
    struct ip *ip;
    struct icmp *icmp;
    struct timeval tv;
    char send_buf[PACKET_LEN];
    char recv_buf[BUFFER_SIZE];
    u_int dst_ip;
    int request;
    int rcv_len;
    int cnt = 0;
    char str[32];
    fd_set readfd;
    int i;

    if (argc != 2)
    {
        printf("Usage : %s <Address>\n", argv[0]);
        exit(0);
    }
```

- 실습에서 만들어야 하는 메시지 형태

Type = 8	Code = 0	checksum
id = process id		seq = sequence number

- ICMP 패킷 생성

```c
void make_icmp_packet(struct icmp *icmp, int len, int n)
{
    memset((char *)icmp, 0, len);
    icmp->icmp_type = ICMP_ECHO;
    icmp->icmp_code = 0;
    icmp->icmp_id = getpid();
    icmp->icmp_seq = n;
    gettimeofday((struct timeval *)icmp->icmp_data, NULL);

    icmp->icmp_cksum = 0;
    icmp->icmp_cksum = checksum((u_short *)icmp, len);
}
```

Raw Socket 구현

```c
#include <netinet/ip.h>      // struct ip*
#include <netinet/ip_icmp.h> // stuct icmp*
#include <netdb.h>           // gethostbyname

#define PACKET_LEN 64
#define BUFFER_SIZE 500
#define MAX_LOOP 5
```

```c
int main(int argc, char *argv[])
{
    int ping_sock; // socket discriptor
    struct hostent *dst_net;
    struct sockaddr_in dst_sa;
    struct ip *ip;
    struct icmp *icmp;
    struct timeval tv;
    char send_buf[PACKET_LEN];
    char recv_buf[BUFFER_SIZE];
    u_int dst_ip;
    int request;
    int rcv_len;
    int cnt = 0;
    char str[32];
    fd_set readfd;
    int i;

    if (argc != 2)
    {
        printf("Usage : %s <Address>\n", argv[0]);
        exit(0);
    }
```

```c
struct icmp
{
  uint8_t  icmp_type; /* type of message, see below */
  uint8_t  icmp_code; /* type sub code */
  uint16_t icmp_cksum;  /* ones complement checksum of struct */
  union
  {
    unsigned char ih_pptr;  /* ICMP_PARAMPROB */
    struct in_addr ih_gwaddr; /* gateway address */
    struct ih_idseq   /* echo datagram */
    {
      uint16_t icd_id;
      uint16_t icd_seq;
    } ih_idseq;
    uint32_t ih_void;

    /* ICMP_UNREACH_NEEDFRAG -- Path MTU Discovery (RFC1191) */
    struct ih_pmtu
    {
      uint16_t ipm_void;
      uint16_t ipm_nextmtu;
    } ih_pmtu;

    struct ih_rtradv
    {
      uint8_t irt_num_addrs;
      uint8_t irt_wpa;
      uint16_t irt_lifetime;
    } ih_rtradv;
  } icmp_hun;
#define icmp_pptr icmp_hun.ih_pptr
#define icmp_gwaddr icmp_hun.ih_gwaddr
#define icmp_id   icmp_hun.ih_idseq.icd_id
#define icmp_seq  icmp_hun.ih_idseq.icd_seq
#define icmp_void icmp_hun.ih_void
#define icmp_pmvoid icmp_hun.ih_pmtu.ipm_void
#define icmp_nextmtu  icmp_hun.ih_pmtu.ipm_nextmtu
```

Raw Socket 구현

```c
// IP 주소면 바로 IP 주소로 출력
if (inet_addr(argv[1]) != INADDR_NONE)
{
    printf("PING %s\n", argv[1]);
    dst_ip = inet_addr(argv[1]);
}
// 유효하지 않은 IP 주소거나 도메인 네임이면 실행
else
{
    if ((dst_net = (gethostbyname(argv[1]))) == NULL)
        error_handling("gethostbyname() error!\n");
    else
    {
        inet_ntop(dst_net->h_addrtype, dst_net->h_addr, str, sizeof(str));
        printf("PING %s\n", str);
        dst_ip = inet_addr(str);
    }
}
```

- 입력한 주소를 확인하고 “PING IP주소” 형태로 출력
- 만약 도메인 주소면 inet_ntop() 함수로 IP주소를 binary 형태에서 사람이 알아보기 쉬운 문자열로 변환

Raw Socket 구현

```
memset((char *)&_______, 0, sizeof(struct _______)); // 1
dst_sa.sin_family = _______;                          // 1
dst_sa.sin_addr.s_addr = _______;                     // 1

ping_sock = socket(_______, _______, _______); // 2
```

- 1. memset()으로 dst_sa 초기화
- 2. ping socket 생성 : 프로토콜 정보로 ICMP 사용

Raw Socket 구현

```
for (cnt = 0; cnt < MAX_LOOP; cnt++)
{
    //send icmp packet
    make_icmp_packet((struct icmp *)send_buf, PACKET_LEN, cnt);
    if (sendto(_______, (char *)&_______, _______, 0, (struct sockaddr *)&_______, sizeof(_______)) < 0) // 3
        error_handling("sendto() error!\n");

    // read icmp message
    rcv_len = recvfrom(_______, _______, _______, 0, NULL, NULL); // 4
    if (rcv_len < 0)
        error_handling("recvfrom() error!\n");

    ip = (struct ip *)recv_buf;
    icmp = (struct icmp *)(recv_buf + (ip->ip_hl * 4));
```

- 3. 패킷 송신 : sendto()로 send_buf를 PACKET_LEN만큼 전송
- 4. 패킷 수신 : recvfrom() 함수로 recv_buf에 BUFFER_SIZE만큼 저장

Raw Socket 구현

- 5 & 6. ICMP 패킷의 sequence number, TTL 출력하기

```c
        // print result
        printf("%d bytes from %s : ", ntohs(ip->ip_len) - ip->ip_hl * 4, inet_ntoa(ip->ip_src));
        printf("icmp_seq : %d ", icmp->_______ + 1); // 5
        printf("TTL = %d ", ip->_______);            // 6

        gettimeofday(&tv, NULL);
        printf("time = %.3f ms ", tv_sub(&tv, (struct timeval *)icmp->icmp_data));
        printf("\n");

        sleep(1);
    }

    return 0;
}
```

❖ 참고자료 - struct icmp

```c
struct icmp
{
    uint8_t  icmp_type; /* type of message, see below */
    uint8_t  icmp_code; /* type sub code */
    uint16_t icmp_cksum;    /* ones complement checksum of struct */
    uint16_t icd_id;    /* identifier */
    uint16_t icd_seq; /* sequence number */
}
```

❖ 참고자료 - struct ip

```c
/*
 * Structure of an internet header, naked of options.
 */
struct ip
  {
#if __BYTE_ORDER == __LITTLE_ENDIAN
    unsigned int ip_hl:4;   /* header length */
    unsigned int ip_v:4;    /* version */
#endif
#if __BYTE_ORDER == __BIG_ENDIAN
    unsigned int ip_v:4;    /* version */
    unsigned int ip_hl:4;   /* header length */
#endif
    uint8_t ip_tos;     /* type of service */
    unsigned short ip_len;    /* total length */
    unsigned short ip_id;   /* identification */
    unsigned short ip_off;    /* fragment offset field */
#define IP_RF 0x8000      /* reserved fragment flag */
#define IP_DF 0x4000      /* dont fragment flag */
#define IP_MF 0x2000      /* more fragments flag */
#define IP_OFFMASK 0x1fff   /* mask for fragmenting bits */
    uint8_t ip_ttl;     /* time to live */
    uint8_t ip_p;     /* protocol */
    unsigned short ip_sum;    /* checksum */
    struct in_addr ip_src, ip_dst;  /* source and dest address */
  };
```

과제 #08

- **제출 파일:** hw08_ping.c, 실행 결과 스크린샷 이미지 (jpg, png 등)
- **제출 방법:** '학번_이름_HW08.zip'으로 압축하여 LMS 과제 게시판에 제출
- **제출 기한:** 2021년 1월 8일 (금) 13:59까지

실행 결과 스크린샷 이미지 예시

```
                                    Desktop/08_ping$ gcc hw8_ping_.c -o ping
                                    Desktop/08_ping$ sudo ./ping 127.0.0.1
[sudo] password for
PING 127.0.0.1
64 bytes from 127.0.0.1 : icmp_seq : 1 TTL = 128 time = 0.308 ms
64 bytes from 127.0.0.1 : icmp_seq : 2 TTL = 128 time = 0.457 ms
64 bytes from 127.0.0.1 : icmp_seq : 3 TTL = 128 time = 0.203 ms
64 bytes from 127.0.0.1 : icmp_seq : 4 TTL = 128 time = 0.210 ms
64 bytes from 127.0.0.1 : icmp_seq : 5 TTL = 128 time = 0.213 ms
                                    Desktop/08_ping$ sudo ./ping google.com
PING 216.58.200.78
64 bytes from 216.58.200.78 : icmp_seq : 1 TTL = 114 time = 42.865 ms
64 bytes from 216.58.200.78 : icmp_seq : 2 TTL = 114 time = 45.718 ms
64 bytes from 216.58.200.78 : icmp_seq : 3 TTL = 114 time = 43.147 ms
64 bytes from 216.58.200.78 : icmp_seq : 4 TTL = 114 time = 42.976 ms
64 bytes from 216.58.200.78 : icmp_seq : 5 TTL = 114 time = 42.350 ms
                                    Desktop/08_ping$ sudo ./ping 8.8.8.8
PING 8.8.8.8
64 bytes from 8.8.8.8 : icmp_seq : 1 TTL = 114 time = 42.386 ms
64 bytes from 8.8.8.8 : icmp_seq : 2 TTL = 114 time = 42.491 ms
64 bytes from 8.8.8.8 : icmp_seq : 3 TTL = 114 time = 42.405 ms
64 bytes from 8.8.8.8 : icmp_seq : 4 TTL = 114 time = 42.628 ms
64 bytes from 8.8.8.8 : icmp_seq : 5 TTL = 114 time = 42.515 ms
                                    Desktop/08_ping$ sudo ./ping naver.com
PING 125.209.222.141
^C
```

✓ 반드시 root 권한으로 실행!

✓ root 권한으로 실행하지 않은 경우

```
                                    Desktop/08_ping$ ./ping google.com
PING 216.58.200.78
socket() error!
```

✓ IP주소 및 DNS

- google.com (8.8.8.8 or 8.8.4.4)
- cloudflare-dns.com (1.1.1.1 or 1.0.0.1)
  - 원하는 다른 IP주소 및 DNS를 사용해도 됨

✓ 호스트가 ping에 대한 응답을 지원하지 않는 사이트도 있으므로 주의

```
                                    Desktop/08_ping$ sudo ./ping naver.com
PING 125.209.222.141
^C
```